

Documenti autorizzati: due fogli manoscritti A4 fronte retro.

Dovete fare A SCELTA o l'esercizio 4 o l'esercizio 5 MA NON entrambi, in tal caso verrà corretto solo il 4

1 Esercizio 1

<pre>int V=0; //Variabile condivisa</pre>	
<pre>--Process P int lp=0; while(true){ p1: Sezione Non Critica p2: V=a; p3: lp=V; p4: while(lp>0){ p5: lp=lp-a;} p6: if(lp<0){goto p2;} p7: Sezione Critica p8: lp=0; }</pre>	<pre>--Process Q int lq=0; while(true){ q1: Sezione Non Critica q2: V=b; q3: lq=V; q4: while(lq>0){ q5: lq=lq-b;} q6: if(lq<0){goto q2;} q7: Sezione Critica q8: lq=0; }</pre>

Figura 1: Algoritmo di mutua esclusione

Consideriamo l'algoritmo di mutua esclusione proposto alla figura 1 dove l'istruzione `goto p` manda il programma alla riga `p` e dove `a` e `b` rappresentano valore intere positive non fissate.

1. Disegnare l'inizio dello diagramma degli stati per questo algoritmo (massimo 10 stati) prendendo 1 come valore di `a` e 2 come valore di `b`.
2. Se scegliamo 1 come valore di `a` e 2 come valore di `b`. Questo algoritmo verifica la proprietà di mutua esclusione? Giustificare.
3. Esistono dei valori per `a` e `b` per fare in modo che l'algoritmo verifica la proprietà di mutua esclusione? Giustificare.
4. Se scegliamo 2 come valore di `a` e 3 come valore di `b`, spiegare perché questo algoritmo non verifica la proprietà di assenza di deadlock. (Ricordiamo che c'è deadlock se i due processi rimangono bloccati per sempre fra la Sezione Non Critica e la Sezione Critica).

2 Esercizio 2

Consideriamo una scatola che può contenere al massimo $P > 0$ cioccolatini. Abbiamo un pasticciere che in ciclo riempie la scatola con dei cioccolatini, ma la riempie direttamente con P cioccolatini e quindi lo può fare solo quando la scatola è vuota. Abbiamo anche dei mangiatori di cioccolatini che in ciclo prendono un cioccolatino dalla scatola e lo mangiano e se la scatola è vuota, aspettano che ci sia di nuovo cioccolatini.

Proponete in JAVA, C o pseudo-codice una modellazione di questo sistema, prendendo in considerazione che vogliamo un processo per il pasticciere e un processo per i mangiatori e che prenderemo tre mangiatori e P uguale a 5 e che vogliamo evitare le attese attive.

3 Esercizio 3

Consideriamo l'algoritmo con message passing proposto sulla figura 2. Lo scopo di questo algoritmo è di garantire la mutua esclusione per $N > 1$ processi. In questo algoritmo, la variabile `token` è inizializzata a `true` per il processo `P0` e a `false` per tutti gli altri processi. Ricordiamo che in un algoritmo con message passing, due entità si trovano sullo stesso sito: l'entità principale e l'entità responsabile della recezione dei messaggi. Nel algoritmo, le parte di codice fra parentesi quadrate (`[ ... ]`) si eseguono in un modo atomico su lo stesso sito (ma non necessariamente su diversi siti).

1. Descrivere in qualche righe come funziona questo algoritmo.

```

--Process Pi per i in {0,...,N-1}
  boolean token=false; //se i non vale 0
  boolean token=true; //se i vale 0
  boolean CS=false:

  while(true){
p1:   Sezione Non Critica
p2:   [CS=true;]
p3:   while(token==false){
p4:     send(req)->P((i+(N-1))%N);}
p5:   Sezione Critica
p6:   [CS=false;]
  }

--Gestione dei messaggi di Pi i in {0,...,N-1}
case (req):
  [if(CS==false and token==true){
    send(tok)->P((i+1)%N);
    token=false;
  }else if(token==false){
    send(req)->P((i+(N-1))%N);
  }]
case(tok):
  [token=true;]

```

Figura 2: Algoritmo di mutua esclusione con message passing

2. Questo algoritmo verifica la mutua esclusione? Giustificare.
3. Questo algoritmo verifica l'assenza di starvation? Giustificare.
4. Come modificare l'algoritmo per autorizzare al massimo due processi in sezione critica ad ogni istante?

4 Esercizio 4

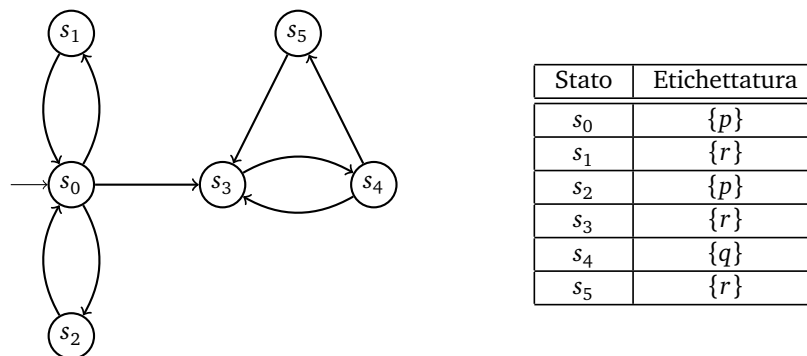


Figura 3: Struttura di Kripke $\mathcal{S}_1$

Consideriamo la struttura di Kripke $\mathcal{S}_1$ rappresentata sulla figura 3 (la tabella a destra indica per ogni stato l'insieme delle proposte atomiche associate) su l'insieme di proposte atomiche $PA = \{p, q, r\}$. Per ciascuna formula ϕ di LTL che seguono, dire (giustificando) se sono soddisfatte dalla struttura (i.e. se abbiamo $S_1 \models \phi$). 1. $\phi = (p \text{ U } r)$ 2. $\phi = (\text{G}p \text{ or } \text{F}r)$ 3. $\phi = (\text{G}\text{F}p)$ 4. $\phi = (\text{G}\text{F}q) \Rightarrow (\text{G}\text{F}r)$

5 Esercizio 5

In una città, esiste una trattoria con una sala d'attesa con 5 sedie ed una stanza con un tavolo per una sola persona. Ovviamente non è possibile avere più di un cliente nella stanza col tavolo ne più di 5 clienti nella sala d'attesa. Il comportamento di un cliente è il seguente: se c'è posto nella sala d'attesa entra, altrimenti aspetta che ci sia posto; poi aspetta che il tavolo sia libero ed entra nella stanza col tavolo e mangia; dopo esce dalla trattoria. Proponete in JAVA, C o pseudo-codice una modellazione di questo sistema, prendendo in considerazione che vogliamo un processo per cliente e consideriamo che ci siano 10 clienti.